

Lab 1

One Problem, Four Paradigms: Implementing

1. Objectives

By the end of this lab, you will be able to:

- Implement the same program logic in procedural, object-oriented, functional, and purely-typed functional styles.
- Identify, in your own code, where state lives and whether it is mutated, hidden, or avoided altogether.
- Compile and run programs written in C, Java, JavaScript (Node.js), and Haskell from the command line.
- Explain, in your own words, how the choice of paradigm changed the shape of your solution.

2. Prerequisites

Make sure the following tools are installed and working before you start. For each, run the version check shown and confirm you get a sensible output (not “command not found”).

Tool	Check with	Where to get it
C compiler	<code>gcc --version</code>	gcc.gnu.org or via your OS package manager (MinGW on Windows, Xcode CLT on macOS, apt on Linux)
Java (JDK)	<code>java -version</code> and <code>javac -version</code>	Install a JDK (e.g. Temurin/OpenJDK 17+)
Node.js	<code>node -v</code>	nodejs.org (LTS version)
Haskell (GHC)	<code>ghc --version</code>	Install via GHCup: get-ghcup.haskell.org , then restart your terminal

3. The Problem

You will write a program that asks the user 10 yes/no questions, then decides whether the user is more of an “Extrovert” or an “Introvert” based on the answers.

Decision rule:

- Questions 1–5 count toward the “Extrovert” score if answered True.
- Questions 6–10 count toward the “Introvert” score if answered True.
- Whichever score is higher wins. If the two scores are equal, print “Balanced (Ambivert)”.

Use exactly these 10 questions, in this order, in every version of your program:

#	Question	Answer (T/F)
1	I enjoy going to parties with lots of people.	
2	I find it easy to start conversations with strangers.	
3	I feel comfortable being the center of attention.	
4	I prefer working in a group rather than alone.	
5	I feel more energized after spending time with other people.	
6	I feel more energized after spending time alone.	
7	I prefer listening over talking in group discussions.	
8	I need quiet time to recharge after socializing.	
9	I prefer a quiet night in over a big social event.	
10	I think carefully before speaking in a group.	

4. What You Must Submit

Write four complete, working programs that all implement the exact same problem above:

4.1 Procedural version — C

Write this using only free-standing functions and global or locally-passed variables. Do not use a struct-with-functions style — the point of this version is that your data and your functions are NOT bundled together.

Guidance

Think in three separate pieces: (1) something that stores the 10 answers, (2) a function that counts how many answers in a range are true, (3) a function that applies the decision rule. Nothing should stop another part of the program from reading or overwriting your answers directly — that is expected and is exactly what makes this version procedural rather than object-oriented.

Compile/Run

```
gcc lab1.c -o lab1 .
./lab1
```

4.2 Object-oriented version — Java

Write a class that has the 10 answers as a private attribute, and public methods to record an answer, ask all the questions, and decide the result. Nothing outside the class should be able to read or modify the answers directly.

Guidance

Ask yourself: what belongs inside the object, and what stays outside? The questions themselves, the stored answers, and the decision logic belong inside your class. The part that prints menus and reads from the keyboard can live in main, calling your object's methods.

Compile/Run

```
javac lab1.java  
java lab1
```

4.3 Functional version — JavaScript (Node.js)

Write this so your decision logic contains no mutation at all — no variable that gets reassigned after it is first set. The scoring and decision functions should be pure: same input always gives the same output, with no side effects.

Guidance

Separate the impure part (reading from the keyboard, which necessarily has side effects) from the pure part (deciding Extrovert/Introvert from a completed list of answers). Your pure functions should be easy to test on their own, without running the whole program or typing anything.

Compile/Run

```
node lab1.js
```

4.4 Purely-typed functional version — Haskell

Write this so that every function doing the actual scoring and deciding has a type signature with no IO in it — only your input-reading function is allowed to mention IO. The goal is for the compiler itself to guarantee purity, not just your own discipline.

Guidance

Split your program into two halves: a pure half (types like `[Bool] -> Int` and `[Bool] -> String`, with no IO anywhere in the signature) that does the counting and decision, and a thin IO half that only asks questions and reads answers. If you can compile the pure half without touching the IO half at all, you have done this correctly.

Compile/Run

```
source ~/.ghcup/env  
ghc lab.hs -o lab  
./lab
```

5. Recommended Order of Work

1. Write and test the C version first — it is the most direct translation of the decision rule into code.
2. Write the Java version next. Reuse the same decision logic, but move the data inside a class.
3. Write the JavaScript version next, once you are comfortable with the logic, since it requires a shift in thinking (no stored, mutable state).
4. Write the Haskell version last — it builds on the same “no mutation” idea as JavaScript, but adds strict typing and a stricter separation between pure and IO code.
5. For each version, test it at least twice: once answering mostly “t” to the first five questions, and once answering mostly “f” to the last five, to confirm both outcomes work.

6. Reflection (short answers, submit with your code)

After finishing all four versions, answer the following in 3–5 sentences total per question:

1. In your C version, what would stop a different part of the program from corrupting your answers array? What does this tell you about procedural programming?
2. In your Java version, what specifically would break if you made your answers field public instead of private?
3. In your JavaScript version, point to one line where you might have been tempted to mutate a variable instead of creating a new value. How did you avoid it?
4. In your Haskell version, look at the type signature of your decision function. How does it prove, without you having to trust your own discipline, that the function cannot secretly print something or read input?

Before you start coding

Re-read Chapter 1, Section 2 (Classification) and Section on Core Concepts. Ask yourself, for each version you write: where does state live, is it mutated, what is hidden, and how do the pieces compose? If you can answer these four questions about your own code, you have understood the paradigm — not just completed the exercise.